

Jenkins-03 – Task 1

1. Create one Jenkins job using the below code and create three stages.

- stage1: Git clone to download the source code.
- stage2: Sonarqube Integration to check the quality of code.
- stage3: Slack Integration to send the alerts to slack.

Source Code: <https://github.com/betawins/VProfile-1.git>

Jenkins Pipeline with Git, SonarQube and Slack Integration

Objective

The objective of this task is to create a Jenkins Pipeline with three stages:

Stage 1 → Git Clone

Stage 2 → SonarQube Code Quality Analysis

Stage 3 → Slack Notification

Source repository:

<https://github.com/betawins/VProfile-1.git>

Branch:

master

Architecture

GitHub

|

| Clone Source Code

v

Jenkins Server

|

| SonarScanner

v

SonarQube Server

|

| Code Quality Report
v
Jenkins
|
| Build Notification
v
Slack #jenkins-alerts

Environment Used

Jenkins Server

OS : Amazon Linux 2023
Java : Java 21
Maven : Maven 3.8.4
RAM : 4 GB
Swap : 2 GB
Jenkins Port: 8081

SonarQube Server

OS : Amazon Linux 2023
Java : Java 21
RAM : 4 GB
Storage : 30 GB
SonarQube : Community Build
Port : 9000

Database

PostgreSQL : 15
Database : sonarqube
DB User : sonar

Slack

Workspace : Chandrashekar-DevOps
Channel : #jenkins-alerts

Implementation Steps

Created a dedicated SonarQube EC2 instance.

An Amazon Linux 2023 EC2 instance was launched for SonarQube with 2 vCPUs and 4 GB

RAM. The root EBS volume was increased from 20 GB to 30 GB.

After increasing the EBS volume, the Linux partition and XFS filesystem were expanded using:

```
growpart /dev/nvme0n1 1
```

```
xfs_growfs -d /
```

Verification:

```
lsblk
```

1. `df -hT /`

Screenshot: EC2 SonarQube server and 30 GB storage.

Installed Java 21 on the SonarQube server.

```
dnf install java-21-amazon-corretto-headless -y
```

2. `java -version`

Java 21 installation was verified successfully.

Screenshot: `java -version`.

Installed PostgreSQL 15.

```
dnf install postgresql15 postgresql15-server -y
```

```
postgresql-setup --initdb
```

```
systemctl enable postgresql
```

```
systemctl start postgresql
```

3. `systemctl status postgresql`

PostgreSQL was verified as:

Active: active (running)

Screenshot: PostgreSQL service status.

Created SonarQube database and database user.

PostgreSQL was opened using:

```
sudo -u postgres psql
```

The following SQL commands were executed:

```
CREATE USER sonar WITH PASSWORD '*****';
```

```
CREATE DATABASE sonarqube OWNER sonar;
```

```
GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonar;
```

Database verification:

```
\l
```

Output confirmed:

Database : sonarqube

4. Owner : sonar

Screenshot: PostgreSQL database list.

Configured Linux kernel settings required by SonarQube.

Created:

```
/etc/sysctl.d/99-sonarqube.conf
```

Added:

```
vm.max_map_count=524288
```

```
fs.file-max=131072
```

Applied configuration:

```
sysctl --system
```

Verified:

```
sysctl vm.max_map_count
```

5. `sysctl fs.file-max`

Screenshot: sysctl verification.

Created a dedicated Linux user for SonarQube.

```
useradd -r -d /opt/sonarqube -s /bin/bash sonarqube
```

6. `id sonarqube`

Screenshot: SonarQube Linux user.

Downloaded and installed SonarQube.

```
dnf install wget unzip -y
```

```
cd /opt
```

```
wget <SonarQube-download-url>
```

```
unzip sonarqube-26.8.0.126808.zip
```

```
mv sonarqube-26.8.0.126808 sonarqube
```

7. `chown -R sonarqube:sonarqube /opt/sonarqube`

Verified:

```
ls -ld /opt/sonarqube
```

Screenshot: SonarQube directory ownership.

Configured SonarQube to use PostgreSQL.

Edited:

```
/opt/sonarqube/conf/sonar.properties
```

Added:

```
sonar.jdbc.username=sonar
```

```
sonar.jdbc.password=*****
```

8. `sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube`

PostgreSQL authentication was changed from `ident` to:

```
scram-sha-256
```

in:

```
/var/lib/pgsql/data/pg_hba.conf
```

PostgreSQL was restarted:
systemctl restart postgresql
Database connection was tested:
psql -h localhost -U sonar -d sonarqube
Verification:
\conninfo

Screenshot: Successful database connection.

Created SonarQube systemd service.

Created:
/etc/systemd/system/sonarqube.service
SonarQube was enabled and started:
systemctl daemon-reload
systemctl enable sonarqube
systemctl start sonarqube

9. systemctl status sonarqube

SonarQube showed:

Active: active (running)

Port verification:

ss -ltnp | grep 9000

Screenshot: SonarQube service running.

Opened SonarQube UI.

Security Group port 9000 was allowed.

SonarQube was opened using:

http://<SONARQUBE-PUBLIC-IP>:9000

Initial credentials:

Username: admin

10. Password: admin

The default password was changed during first login.

Screenshot: SonarQube dashboard.

Generated a SonarQube token for Jenkins.

In SonarQube:

My Account

→ Security

11. → Generate Token

Token name:

Jenkins-03

The generated token was copied securely.

Screenshot: SonarQube token generation page.
Do not expose the actual token in the documentation.

Installed SonarQube Scanner plugin in Jenkins.

In Jenkins:

Manage Jenkins

→ Plugins

→ Available Plugins

→ SonarQube Scanner

12. → Install

Screenshot: Plugin installation successful.

Stored the SonarQube token in Jenkins Credentials.

Path:

Manage Jenkins

→ Credentials

→ System

→ Global Credentials

→ Add Credentials

Configuration:

Kind : Secret text

ID : sonar-token

13. Description : SonarQube Token

Screenshot: `sonar-token` credential.

Connected Jenkins to the SonarQube server.

Path:

Manage Jenkins

→ System

→ SonarQube Servers

Configuration:

Name : SonarQube-Server

Server URL : http://172.31.47.228:9000

14. Authentication: sonar-token

The private IP was used because Jenkins and SonarQube are in the same AWS VPC.

Screenshot: Jenkins SonarQube configuration.

Configured SonarScanner tool in Jenkins.

Path:

Manage Jenkins

→ Tools

→ SonarQube Scanner installations

Configuration:

Name : SonarScanner

15. Install automatically: Enabled

Screenshot: SonarScanner tool configuration.

16. **Tested connectivity from Jenkins to SonarQube.**

On the Jenkins EC2 server:

`curl -s http://172.31.47.228:9000/api/system/status`

Successful output:

`"status": "UP"`

Screenshot: SonarQube connectivity test.

Created Jenkins Pipeline job.

In Jenkins:

New Item

→ Jenkins-03-VProfile

→ Pipeline

Stage 1 was first tested independently using Git clone.

```
stage('Git Clone') {
```

```
  steps {
```

```
    git branch: 'master',
```

```
    url: 'https://github.com/betawins/VProfile-1.git'
```

```
  }
```

17. }

Result:

Finished: SUCCESS

Screenshot: Git Clone successful.

Added SonarQube Analysis stage.

SonarScanner was executed using:

```
stage('SonarQube Analysis') {
```

```
  steps {
```

```
    script {
```

```
      def scannerHome = tool 'SonarScanner'
```

```
      withSonarQubeEnv('SonarQube-Server') {
```

```
        sh """
```

```
          ${scannerHome}/bin/sonar-scanner \
```

```
            -Dsonar.projectKey=VProfile-1 \
```

```

        -Dsonar.projectName=VProfile-1 \
        -Dsonar.sources=src
    """"
    }
}
}
}
}

```

The final successful scan showed:

ANALYSIS SUCCESSFUL

SonarScanner Engine completed successfully

18. EXECUTION SUCCESS

Screenshot: Successful SonarQube console output.

Resolved Jenkins memory issue.

Initially the Jenkins server used a **t3.micro** with approximately 1 GB RAM.

During SonarQube analysis, the JavaScript analyzer failed with:

The bridge server is unresponsive.

It might be because you don't have enough memory.

A 2 GB swap file was first created:

```
sudo fallocate -l 2G /swapfile
```

```
sudo chmod 600 /swapfile
```

```
sudo mkswap /swapfile
```

```
sudo swapon /swapfile
```

Permanent swap entry:

```
echo 'swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Verification:

```
free -h
```

19. swapon --show

The Jenkins EC2 instance was then resized to approximately **4 GB RAM**, after which SonarQube completed successfully.

Screenshot: **free -h** showing 4 GB RAM + 2 GB swap.

20. Created Slack workspace and channel.

Slack workspace:

Chandrashekar-DevOps

Notification channel:

#jenkins-alerts

A custom Slack app named:

Jenkins

was created.

Screenshot: Slack workspace/channel.

21. Generated Slack Bot OAuth Token.

Slack app permissions were configured and the **Bot User OAuth Token** beginning with:
xoxb-
was generated.

The actual token was not stored in the pipeline.

Screenshot: OAuth page without exposing the token.

Stored Slack token in Jenkins Credentials.

In Jenkins:

Manage Jenkins

→ Credentials

→ Global Credentials

→ Add Credentials

Configuration:

Kind : Secret text

ID : slack-token

22. Description : Slack Jenkins Token

Screenshot: Slack Jenkins Token credential.

Configured Slack in Jenkins.

Path:

Manage Jenkins

→ System

→ Slack

Configuration:

Workspace : Chandrashekar-DevOps

Credential : Slack Jenkins Token

Default channel : #jenkins-alerts

23. Custom Slack app bot user : Enabled

Initially Jenkins returned:

not_in_channel

The Jenkins bot was then invited to **#jenkins-alerts**.

After that:

Test Connection → Success

Screenshot: Slack Test Connection Success.

Added Slack Notification stage.

The final Slack stage was:

```
stage('Slack Notification') {
```

```
  steps {
```

```
    slackSend(
```

```

        channel: '#jenkins-alerts',
        color: 'good',
        message: "✅ Jenkins Job SUCCESS: env.JOB_NAME{env.BUILD_NUMBER}"
    )
}
}
A failure notification was also added:
post {
    failure {
        slackSend(
            channel: '#jenkins-alerts',
            color: 'danger',
            message: "❌ Jenkins Job FAILED: env.JOB_NAME{env.BUILD_NUMBER}"
        )
    }
}
24. }

```

Final Jenkins Pipeline.

```

pipeline {
    agent {
        label 'linux'
    }

    stages {

        stage('Git Clone') {
            steps {
                git branch: 'master',
                    url: 'https://github.com/betawins/VProfile-1.git'
            }
        }

        stage('SonarQube Analysis') {
            steps {
                script {
                    def scannerHome = tool 'SonarScanner'

```

```

        withSonarQubeEnv('SonarQube-Server') {
            sh """
            ${scannerHome}/bin/sonar-scanner \
            -Dsonar.projectKey=VProfile-1 \
            -Dsonar.projectName=VProfile-1 \
            -Dsonar.sources=src
            """
        }
    }
}

stage('Slack Notification') {
    steps {
        slackSend(
            channel: '#jenkins-alerts',
            color: 'good',
            message: "✅ Jenkins Job SUCCESS: env.JOB_NAME#{env.BUILD_NUMBER}"
        )
    }
}

post {
    failure {
        slackSend(
            channel: '#jenkins-alerts',
            color: 'danger',
            message: "❌ Jenkins Job FAILED: env.JOB_NAME#{env.BUILD_NUMBER}"
        )
    }
}

25. }

```

Final Result.

The final pipeline completed successfully:

Git Clone → SUCCESS ✓

SonarQube Analysis → SUCCESS ✓

Slack Notification → SUCCESS ✓

Overall Jenkins Build → SUCCESS ✓

SonarQube console output:

ANALYSIS SUCCESSFUL

26. EXECUTION SUCCESS

Jenkins:

Finished: SUCCESS

Slack received the successful Jenkins build notification.

Screenshots: Jenkins final success + SonarQube results + Slack notification.

Note About SonarQube Quality Gate

The SonarQube analysis itself completed successfully, but the project displayed:

Quality Gate → FAILED

This means the Jenkins-to-SonarQube integration worked correctly, but the application source code did not satisfy one or more configured SonarQube quality conditions.

For this assignment, the required **Git Clone + SonarQube Integration + Slack Integration** was completed successfully. `waitForQualityGate()` was not required by the stated task.

Conclusion

Jenkins-03 Task 1 successfully demonstrated a CI workflow integrating **GitHub, Jenkins, SonarQube, PostgreSQL and Slack**.

The final workflow was:

Developer / GitHub



Jenkins Git Clone



SonarQube Static Code Analysis



Code Quality Report



Slack Build Notification

This provides automated source-code retrieval, code-quality analysis, and real-time team notification as part of a Jenkins CI pipeline.

```
aws [Search] [Alt+S] [United States (N. Virginia) chandrashekar (986881030460) chandrashekar]

[root@ip-172-31-47-228 ~]# sudo -u postgres psql
could not change directory to "/root": Permission denied
psql (15.19)
Type "help" for help.

postgres=# CREATE USER sonar WITH PASSWORD 'Sonar@123';
CREATE ROLE
postgres=# CREATE DATABASE sonarqube OWNER sonar;
CREATE DATABASE
postgres=# GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonar;
GRANT
postgres=# \l

      List of databases
  Name | Owner | Encoding | Collate | Ctype | ICU Locale | Locale Provider | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----
postgres | postgres | UTF8 | C.UTF-8 | C.UTF-8 | | libc | |
sonarqube | sonar | UTF8 | C.UTF-8 | C.UTF-8 | | libc | | =Tc/sonar +
template0 | postgres | UTF8 | C.UTF-8 | C.UTF-8 | | libc | | =c/postgres +
template1 | postgres | UTF8 | C.UTF-8 | C.UTF-8 | | libc | | postgres=CtC/postgres +
          (4 rows)

postgres=# \q
[root@ip-172-31-47-228 ~]# cat <<EOF > /etc/sysctl.d/99-sonarqube.conf
vm.max_map_count=524288
fs.file-max=131072
EOF
[root@ip-172-31-47-228 ~]# sysctl --system
* Applying /etc/sysctl.d/00-defaults.conf ...
kernel.printk = 8 4 1 7
kernel.panic = 5
net.ipv4.neigh.default.gc_thresh1 = 0
```

i-061ba756c33a7c9ec (SonarQube-Server)

PublicIPs: 18.209.171.81 PrivateIPs: 172.31.47.228

```
aws [Search] [Alt+S] [Ask Amazon Q] [United States (N. Virginia) chandrashekar (986881030460) chandrashekar]

inflating: sonarqube-26.8.0.126808/web/js/useDebounceValue-DLT029IN.js
inflating: sonarqube-26.8.0.126808/web/js/useResizeObserver-B2M7rHiU.js
inflating: sonarqube-26.8.0.126808/web/js/utills-D9Kughli.js
inflating: sonarqube-26.8.0.126808/web/js/vendor-HweZbVKI.js
inflating: sonarqube-26.8.0.126808/web/js/web-api-GW4WjSp.js
inflating: sonarqube-26.8.0.126808/web/js/web-api-D96cryt6.js
inflating: sonarqube-26.8.0.126808/web/js/withLanguages-D9dfsROE.js
inflating: sonarqube-26.8.0.126808/web/js/withMetricsContext-CkmdqVU1.js
inflating: sonarqube-26.8.0.126808/web/mstile-512x512.png
inflating: sonarqube-26.8.0.126808/web/package.json
inflating: sonarqube-26.8.0.126808/web/robots.txt
inflating: sonarqube-26.8.0.126808/web/vendor.LICENSE.txt
creating: sonarqube-26.8.0.126808/web/WEB-INF/classes/
creating: sonarqube-26.8.0.126808/web/WEB-INF/classes/com/
creating: sonarqube-26.8.0.126808/web/WEB-INF/classes/com/sonarsource/
inflating: sonarqube-26.8.0.126808/web/WEB-INF/classes/com/sonarsource/branding
creating: sonarqube-26.8.0.126808/lib/jdbc/
creating: sonarqube-26.8.0.126808/lib/jdbc/mssql/
inflating: sonarqube-26.8.0.126808/lib/jdbc/mssql/mssql-jdbc-13.4.0.jre11.jar
inflating: sonarqube-26.8.0.126808/lib/jdbc/mssql/msal4j-1.25.0.jar
inflating: sonarqube-26.8.0.126808/lib/jdbc/mssql/slf4j-api-2.0.18.jar
inflating: sonarqube-26.8.0.126808/lib/jdbc/mssql/azure-json-1.4.0.jar
creating: sonarqube-26.8.0.126808/lib/jdbc/postgresql/
inflating: sonarqube-26.8.0.126808/lib/jdbc/postgresql/postgresql-42.7.11.jar
creating: sonarqube-26.8.0.126808/lib/jdbc/h2/
inflating: sonarqube-26.8.0.126808/lib/jdbc/h2/h2-2.3.232.jar
inflating: sonarqube-26.8.0.126808/lib/sonar-shudowner-26.8.0.126808.jar
creating: sonarqube-26.8.0.126808/elasticsearch/plugins/
[root@ip-172-31-47-228 opt]# mv sonarqube-26.8.0.126808 sonarqube
[root@ip-172-31-47-228 opt]# chown -R sonarqube:sonarqube /opt/sonarqube
[root@ip-172-31-47-228 opt]# ls -ld /opt/sonarqube
drwxr-xr-x. 14 sonarqube sonarqube 16384 Aug 4 20:44 /opt/sonarqube
[root@ip-172-31-47-228 opt]# cp /opt/sonarqube/conf/sonar.properties /opt/sonarqube/conf/sonar.properties.backup
[root@ip-172-31-47-228 opt]#
```

i-061ba756c33a7c9ec (SonarQube-Server)

PublicIPs: 18.209.171.81 PrivateIPs: 172.31.47.228

←→↺🏠

🚦 Not secure18.209.171.81:9000/projects/create

🔍🌟👤

Relaunch to update

🚦 Apps🗂️

📺 YouTube

🔍 Google

📺 12 Beginner Python...

📺 Learn Python

📺 Learn Data Structur...

📺 LeetCode - The Wor...

📺 Movierulz | Watch B...

📺 Slack

1

Keep your instance current and get the **latest SonarQube Community Build**, available now.

2

There's a new version of SonarQube Server available. Upgrade to SonarQube Server and get access to enterprise features. [Learn More](#)

SonarQube
community

[Projects](#) [Issues](#) [Rules](#) [Quality profiles](#) [Quality gates](#) [Administration](#) [More ▾](#)

🔍👤?A

How do you want to create your project?

Do you want to benefit from all of SonarQube Community Build's features (like repository import and Pull Request decoration)?

Create your project from your favorite DevOps platform.

First, you need to set up a DevOps platform configuration.

🏠 Import from Azure DevOps

Setup

📁 Import from Bitbucket Cloud

Setup

📁 Import from Bitbucket Data Center

Set...

🔄 Import from GitHub

Setup

🐙 Import from GitLab

Setup

Are you just testing or have an advanced use-case?

[Create a local project](#)



Get the most out of SonarQube Community Build!

Take advantage of the whole ecosystem by using SonarQube for IDE, a free IDE plugin that helps you find and fix issues easier in your workflow. Connect SonarQube for IDE to SonarQube Community Build to sync rule sets and issue states.

[Learn More](#)

Dismiss

← → ↻ 🏠 ⚠ Not secure 98.92.209.71:8081/manage/pluginManager/updates/ ☆

Apps YouTube Google 12 Beginner Python... Learn Python Learn Data Structur... LeetCode - The Wor... Movierulz | Watch B... Slack

Jenkins / Manage Jenkins ▾ / Plugins

Plugins

- 📁 Updates
- 📦 Available plugins
- ⚙ Installed plugins
- ⚙ Advanced settings
- ☰ Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

SonarQube Scanner ✓ Success

Loading plugin extensions ✓ Success

→ [Go back to the top page](#)
(you can start using the installed plugins right away)

→ ☐ Restart Jenkins when installation is complete and no jobs are running

← → ↻ 🏠 ⚠ Not secure 98.92.209.71:8081/job/Jenkins-03-VProfile/4/console ☆ 🔄 ⚙ 🧑 Relaunch to update

Apps YouTube Google 12 Beginner Python... Learn Python Learn Data Structur... LeetCode - The Wor... Movierulz | Watch B... Slack

Jenkins / Jenkins-03-VProfile ▾ / #4 ▾ 🔍 ⚙ 🧑

🔄 Previous Build

```
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/betawins/VProfile-1.git
> git init /var/lib/jenkins/workspace/Jenkins-03-VProfile # timeout=10
Fetching upstream changes from https://github.com/betawins/VProfile-1.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/VProfile-1.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/betawins/VProfile-1.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision afc137167b41aaaffaeaf82b81aee721ec883901 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f afc137167b41aaaffaeaf82b81aee721ec883901 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git checkout -b master afc137167b41aaaffaeaf82b81aee721ec883901 # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

←

→

↺

🏠

🔒 Not secure

100.56.237.214:8081/job/Jenkins-03-VProfile/

☆

📄

👤

Relaunch to update

📱 Apps

📺 YouTube

🔍 Google

📺 12 Beginner Python...

📺 Learn Python

📺 Learn Data Structur...

📺 LeetCode - The Wor...

📺 Movierulz | Watch B...

📺 Slack

 **Jenkins**

Jenkins-03-VProfile

🔍

⚙️

👤

</> Changes

▶ Build Now

🗑 Delete Pipeline

⚙ Configure

✏ Rename

📡 SonarQube

📁 Stages

❓ Pipeline Syntax

Builds >

🔍 Filter

/

Today

🟢 #8 3:36 PM

📡

▼

SonarQube Quality Gate

VProfile-1

Passed

server-side processing:

Success

Permalinks

- [Last build \(#8\), 4 min 27 sec ago](#)
- [Last stable build \(#8\), 4 min 27 sec ago](#)
- [Last successful build \(#8\), 4 min 27 sec ago](#)
- [Last failed build \(#7\), 29 min ago](#)
- [Last unsuccessful build \(#7\), 29 min ago](#)
- [Last completed build \(#8\), 4 min 27 sec ago](#)

Install App

Manage Distribution

Features

App Home

Agents

Work Object Previews...

Workflow Steps

Org Level Apps

Incoming Webhooks

Interactivity & Shortcuts

Slash Commands

OAuth & Permissions

MCP Servers

New

Event Subscriptions

User ID Translation

App Manifest

Beta Features

Submit to Slack Marketplace

Prepare & Submit

1. Your App Listing

token rotation allows app tokens to automatically expire after they're issued within your app code. [View documentation.](#)

⚠ At least one redirect URL needs to be set below before this app can be opted into token rotation

Opt In

Proof Key for Code Exchange (PKCE)

Enabling this feature lets your app use PKCE during OAuth. This is required if your app redirects to a custom URI scheme like myapp://OAuth, and optional for web-based authentication. See [documentation](#) for more details.

Opt In

OAuth Tokens

You can use these tokens to authenticate your app. [Learn more.](#)

Bot User OAuth Token

xxxb-12233670232864-12199179393267-7dF1ZqiElc7C9wjI736B6nF9

Copy

Access Level: Workspace

Reinstall to Chandrashekar-DevOps

CD Chandrashekar-DevOps

Home

DMs

Activity

More

Admin

Channels

- # all-chandrashekar-devops
- # jenkins-alerts
- # social
- + Add channels

Direct messages

- chandrashekar you
- + Invite people

Unread mentions

Agents & apps

Huddles

Directories

Starred

Drag and drop important stuff here

Find a conversation...

An offer awaits

Search Chandrashekar-DevOps

jenkins-alerts

Invite teammates

Messages

Add canvas

joined #jenkins-alerts.

Today

chandrashekar 9:32 PM

renamed the channel from "new-channel" to "jenkins-alerts"

Jenkins APP 9:48 PM

was added to #jenkins-alerts by chandrashekar.

Only visible to you

Slackbot 9:48 PM

Invites have been sent! Kick back and relax while you wait for them to join.

Only visible to you

@Jenkins is already in this channel.

Jenkins APP 9:50 PM

Slack/Jenkins plugin: you're all set on <http://32.196.117.219:8081/>

Jenkins Job FAILED: Jenkins-03-VProfile #9

B I U S | | | | | | | | | |

Message #jenkins-alerts

+ Aa @ []

Huddles notes and AI assistant apps are turned on for Chandrashekar-DevOps. Learn more

← → ↺ 🏠 ⚠ Not secure 44.195.129.88:8081/job/Jenkins-03-VProfile/

Apps | YouTube | Google | 12 Beginner Python... | Learn Python | Learn Data Structur... | LeetCode - The Wor...

Jenkins / Jenkins-03-VProfile

server-side processing: **Success**

Permalinks

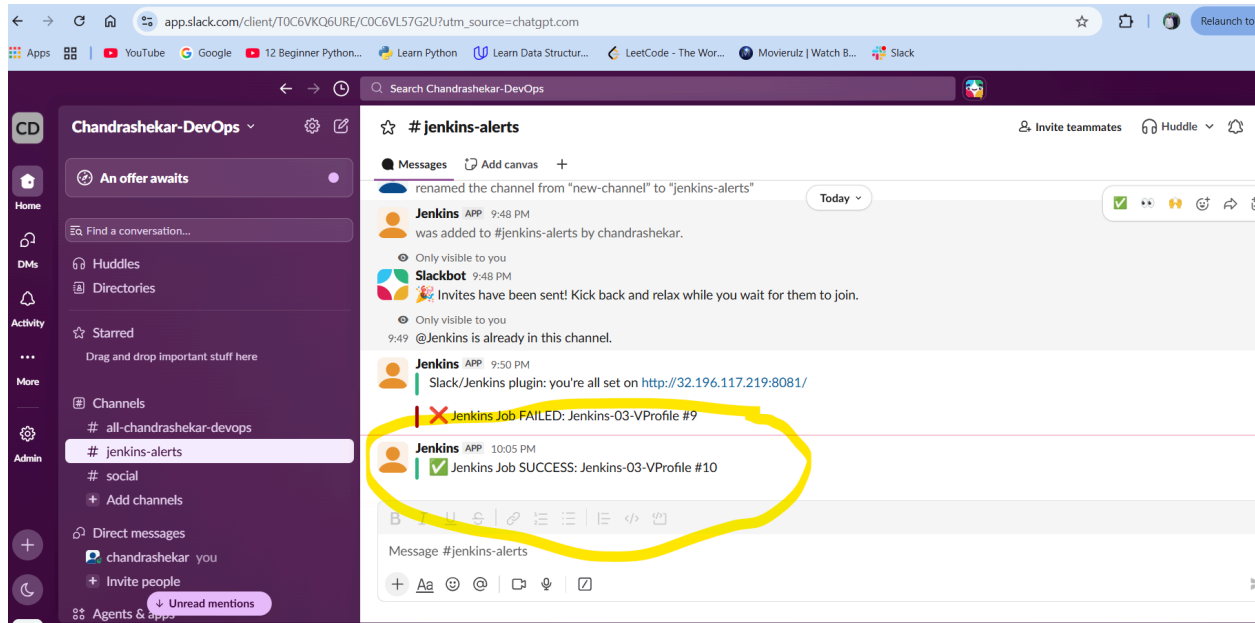
- Last build (#10), 1 min 5 sec ago
- Last stable build (#10), 1 min 5 sec ago
- Last successful build (#10), 1 min 5 sec ago
- Last failed build (#9), 13 min ago
- Last unsuccessful build (#9), 13 min ago
- Last completed build (#10), 1 min 5 sec ago

Builds >

Filter

Today

✓ #10 4:34 PM



2. Create one Jenkins job using the below code and create three stages.

- stage1: Git clone to download the source code.
- stage2: Sonarqube Integration to check the quality of code.
- stage3: Slack Integration to send the alerts to slack.

Source Code: <https://github.com/betawins/hiring-app.git>

Jenkins-03 – Task 2

Jenkins Pipeline with Git, SonarQube and Slack Integration for Hiring-App

Objective

The objective of this task is to create a Jenkins Pipeline with three stages:

Stage 1 → Git Clone

Stage 2 → SonarQube Analysis

Stage 3 → Slack Notification

Source repository:

<https://github.com/betawins/hiring-app.git>

Branch:

main

Jenkins job name:

Jenkins-03-Hiring-App

Architecture

GitHub



Jenkins



Git Clone



SonarQube Analysis



Quality Gate



Slack Notification

Existing Setup Reused from Task 1

The following components were already configured and therefore were reused:

Jenkins Server

SonarQube Server

PostgreSQL Database

SonarQube Scanner Plugin

SonarScanner Tool

SonarQube Token

Slack Plugin

Slack Bot Token

Slack Workspace

Slack Channel

Existing Jenkins configurations used:

SonarQube Server Name : SonarQube-Server

SonarScanner Tool : SonarScanner

Slack Workspace : Chandrashekar-DevOps

Slack Channel : #jenkins-alerts

Slack Credential : slack-token

Jenkins Node Label : linux

Implementation Steps

Step 1 – Create a New Jenkins Pipeline Job

Open Jenkins Dashboard.

Go to:

Jenkins Dashboard

→ New Item

Enter the job name:

Jenkins-03-Hiring-App

Select:

Pipeline

Click:

OK

Screenshot: Jenkins new pipeline job creation.

Step 2 – Configure Stage 1: Git Clone

Go to:

Jenkins-03-Hiring-App

→ Configure

→ Pipeline

Initially, configure only the Git Clone stage.

```
pipeline {  
  agent {  
    label 'linux'  
  }  
  
  stages {  
    stage('Git Clone') {  
      steps {  
        git branch: 'main',  
          url: 'https://github.com/betawins/hiring-app.git'  
      }  
    }  
  }  
}
```

Click:

Save

→ Build Now

Open:

Build

→ Console Output

The console confirmed:

Cloning repository <https://github.com/betawins/hiring-app.git>

Checking out Revision ...

refs/remotes/origin/main

Finished: SUCCESS

Result:

Stage 1 – Git Clone → SUCCESS

Screenshot: Git Clone console output showing **Finished: SUCCESS**.

Step 3 – Add SonarQube Analysis Stage

After verifying Git Clone, edit the same pipeline.

Add Stage 2:

```
stage('SonarQube Analysis') {
    steps {
        script {
            def scannerHome = tool 'SonarScanner'

            withSonarQubeEnv('SonarQube-Server') {
                sh """
                ${scannerHome}/bin/sonar-scanner \
                -Dsonar.projectKey=Hiring-App \
                -Dsonar.projectName=Hiring-App \
                -Dsonar.sources=src
                """
            }
        }
    }
}
```

The important values used were:

Project Key : Hiring-App

Project Name : Hiring-App

Source Path : src

The SonarQube project was kept separate from Task 1 by using a new project key.

Step 4 – Run the Pipeline with SonarQube

Click:

Save

→ Build Now

→ Console Output

The console showed Jenkins successfully communicating with SonarQube:

Communicating with SonarQube Community Build

SonarScanner detected the project:

Project key: Hiring-App

Base dir: /var/lib/jenkins/workspace/Jenkins-03-Hiring-App

SonarQube indexed the source files and completed the analysis.

Important successful output:

ANALYSIS SUCCESSFUL

SonarScanner Engine completed successfully

EXECUTION SUCCESS

Finished: SUCCESS

The analysis took approximately:

Total time: 7 seconds

Result:

Stage 2 – SonarQube Analysis → SUCCESS

Screenshot: Console output showing ANALYSIS SUCCESSFUL and EXECUTION SUCCESS.

Step 5 – Verify Hiring-App in SonarQube

Open the SonarQube dashboard.

Go to:

Projects

→ Hiring-App

SonarQube successfully created the project:

Hiring-App

The Jenkins job also displayed:

SonarQube Quality Gate

Hiring-App → Passed
server-side processing → Success

Result:

SonarQube Analysis → SUCCESS
SonarQube Quality Gate → PASSED

Screenshot: Jenkins page showing:

Hiring-App Passed
server-side processing: Success

This is an important screenshot for Task 2.

Step 6 – Add Slack Notification Stage

Slack was already configured during Task 1.

Existing Slack details:

Workspace : Chandrashekar-DevOps
Channel : #jenkins-alerts
Credential: slack-token

Add the following third stage:

```
stage('Slack Notification') {  
    steps {  
        slackSend(  
            channel: '#jenkins-alerts',  
            color: 'good',  
            message: "✅ Jenkins Job SUCCESS: env.JOB_NAME#{env.BUILD_NUMBER}"  
        )  
    }  
}
```

This sends a successful Jenkins notification to Slack when all pipeline stages complete.

```

withSonarQubeEnv('SonarQube-Server') {
    sh """
    ${scannerHome}/bin/sonar-scanner \
    -Dsonar.projectKey=Hiring-App \
    -Dsonar.projectName=Hiring-App \
    -Dsonar.sources=src
    """
}
}
}

stage('Slack Notification') {
    steps {
        slackSend(
            channel: '#jenkins-alerts',
            color: 'good',
            message: "✅ Jenkins Job SUCCESS: env.JOB_NAME#{env.BUILD_NUMBER}"
        )
    }
}

post {
    failure {
        slackSend(
            channel: '#jenkins-alerts',
            color: 'danger',
            message: "❌ Jenkins Job FAILED: env.JOB_NAME#{env.BUILD_NUMBER}"
        )
    }
}
}

```

Final Execution

The final pipeline executed in this order:

GitHub Hiring-App



Stage 1

Git Clone



SUCCESS



Stage 2

SonarQube Analysis



ANALYSIS SUCCESSFUL



Quality Gate

PASSED



Stage 3

Slack Notification



SUCCESS

Console output confirmed:

ANALYSIS SUCCESSFUL

SonarScanner Engine completed successfully

EXECUTION SUCCESS

Slack stage executed using:

teamDomain: Chandrashekar-DevOps

channel: #jenkins-alerts

color: good

botUser: true

tokenCredentialId: slack-token

Final Jenkins result:

Finished: SUCCESS

Final Results

Component	Result
Git Clone	 SUCCESS
SonarQube Analysis	 SUCCESS
SonarQube Quality Gate	 PASSED
Slack Notification	 SUCCESS
Jenkins Pipeline	 SUCCESS

Screenshots to Add

For the Task 2 document, add screenshots in this order:

1. **Jenkins-03-Hiring-App job**
2. **Git Clone – Finished SUCCESS**
3. **SonarQube – ANALYSIS SUCCESSFUL**
4. **SonarQube Hiring-App project**
5. **Jenkins Quality Gate – Passed**
6. **Slack success notification**
7. **Final Jenkins build – green SUCCESS**

Your final Jenkins screenshot showing:

Hiring-App Passed
server-side processing: Success

is especially good for the final result section.

Conclusion

Jenkins-03 Task 2 was successfully completed by integrating **GitHub, Jenkins, SonarQube and Slack**.

The Jenkins pipeline automatically cloned the Hiring-App source code from GitHub, performed static code-quality analysis using SonarQube, verified that the **Quality Gate Passed**, and sent a successful build notification to Slack.

The final workflow was:

GitHub



Jenkins



Git Clone



SonarQube Analysis



Quality Gate Passed



Slack Notification



Pipeline SUCCESS

app.slack.com/client/T0C6VKQ6URE/C0C6VL57G2U?utm_source=chatgpt.com

Chandrashekar-DevOps

#jenkins-alerts

Messages Add canvas +

Only visible to you

Slackbot 9:48 PM

Invites have been sent! Kick back and relax while you wait for them to join.

Only visible to you

@Jenkins is already in this channel.

Jenkins APP 9:50 PM

Slack/Jenkins plugin: you're all set on <http://32.196.117.219:8081/>

Jenkins Job FAILED: Jenkins-03-VProfile #9

Jenkins APP 10:05 PM

Jenkins Job SUCCESS: Jenkins-03-VProfile #10

Jenkins APP 10:50 PM

Jenkins Job SUCCESS: Jenkins-03-Hiring-App #3

Message #jenkins-alerts

Huddles notes and AI assistant apps are turned on for Chandrashekar-DevOps. [Learn more](#)